

Automated data workflows: downloading, visualisation and reporting

Alari Paulus & Nicolas Reigl

RMO seminar
17 April 2025

Why this session?

Initially to demonstrate [data fetching](#), which expanded with:

- ▶ *'Osakondade andmeanalüüsi ja visualiseerimise keskkonna nõuete kogumine'* (and BO)
- ▶ Follow-up AI course (beyond the introduction) – what people need
- ▶ Personal (random) observations

Main aim: demonstrate possibilities of automated data workflows, discuss current practices and clarify training needs.

Plan for today

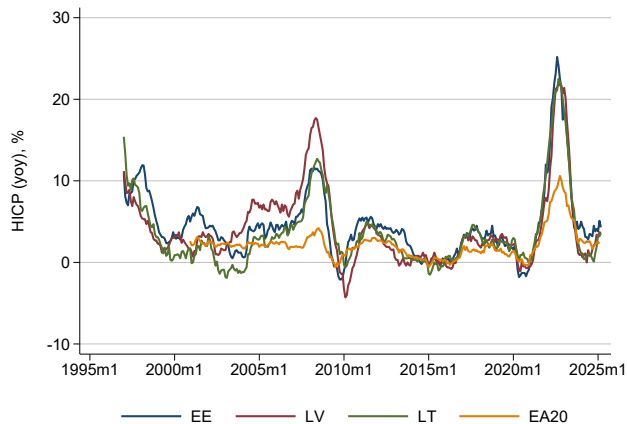
- ▶ Demonstrate **automated** data downloading, visualisation & reporting
 - ▶ learn from each other and share best practices
 - ▶ researchers, economists, analysts, statisticians
- ▶ Semi-technical
 - ▶ live examples/interactive
 - ▶ focus on what is possible and general logic, not how to do it with specific software and commands
 - ▶ example codes shared (Stata, R, Python)
- ▶ Discuss current uses of AI and training needs

Stages of quantitative research/analysis

1. Data fetching
2. *Data restructuring and cleaning*
3. Descriptive analysis and data visualisation
4. *Model estimation and visualisation*
5. Final outputs – papers, reports, slides

Käsk #11: Sina ei pea mitte masina eest tööd ära tegema!

Example 1: Full workflow



Example 1: Stata code

```
// fetch data from Eurostat
sdmxuse data ESTAT, dataset(prc_hicp_manr) dimensions(M.RCH_A.CP00.) clear

// data processing
gen ym = monthly(time, "YM")
format ym %tm
label var value "HICP (yoy), %"

// figure
tw (line value ym if geo == "EE") (line value ym if geo == "LV") ///
   (line value ym if geo == "LT") (line value ym if geo == "EA20"), ///
   legend(rows(1) order(1 "EE" 2 "LV" 3 "LT" 4 "EA20" )) xtitle("")

// export for slides
graph export "hicp_m.pdf", replace
```

General principles of quantitative analysis

1. Transparency
 - ▶ what, how, when
2. Replicability
 - ▶ can be easily redone when needed (always more times than expected!)
3. Consistency and accuracy
 - ▶ avoid errors from (repeated) manual operations, copy/pasting or copy/typing
4. Speed
 - ▶ quicker processing and updating, economies of scale (extensions, scaling-up)

If possible, **avoid** intermediate data processing with Excel etc.

API in a nutshell

- ▶ Software programs can be connected with **APIs** (Application Programming Interface)
- ▶ *API specifications* describe how to build such connections
- ▶ Web APIs allow accessing online resources, e.g. RESTful API
- ▶ Typically use HTTP request messages (GET, POST) in XML or JSON format
- ▶ Data fetching vs web scraping

Online databases with API

- ▶ Main providers of public (macro) statistics offer API
 - ▶ Eurostat, ECB, OECD, IMF, ..., Statistics Estonia
 - ▶ Eesti Pank does **not**
- ▶ Often support **SDMX** (Statistical Data and Metadata eXchange)
 - ▶ ISO standard sponsored by BIS, ECB, ESTAT, ILO, IMF, OECD, UNSD & WB
 - ▶ Structured HTTP GET queries
 - ▶ Supports XML, JSON and CSV formats for metadata and datasets
 - ▶ Metadata: list of datasets (*dataflow*) and structure of datasets (*datastructure*)
- ▶ Stata has a user-written command for SDMX queries (SDMXUSE)

Example 2: SDMX queries to Eurostat (Stata)

► List of datasets (DATAFLOW):

```
sdmxuse dataflow ESTAT, clear
```

► Structure of datasets (DATASTRUCTURE):

```
sdmxuse datastructure ESTAT, dataset(PRC_HICP_MANR) clear
```

► Data:

```
sdmxuse data ESTAT, dataset(PRC_HICP_MANR) dimensions(..CP00.EE) clear
```

Example 2: SDMX queries to Eurostat (manual download in XML)

► List of datasets (DATAFLOW):

`https:`

`//ec.europa.eu/eurostat/api/dissemination/sdmx/2.1/dataflow/ESTAT/all?detail=allstubs`

► Structure of datasets (DATASTRUCTURE):

`https://ec.europa.eu/eurostat/api/dissemination/sdmx/2.1/dataflow/ESTAT/PRC_HICP_MANR/
?references=children`

► Data:

`https://ec.europa.eu/eurostat/api/dissemination/sdmx/2.1/data/PRC_HICP_MANR/..CP00.EE/
all?detail=full`

Example 2: Determine the structure and content of datasets

```
sdmxuse datastructure ESTAT, dataset(PRC_HICP_MANR) clear // dimensions: (freq.unit.coicop.geo)

des // concept, position, code, code_lbl - common across sites
ta concept // coicop, freq, geo, unit - dataset specific
su position // 1-4th in the order of dimension

list if concept == "freq", clean noobs // e.g. M (monthly)

count if concept == "unit" // 700
list if concept == "unit" & regexm(code_lbl, "rate"), clean noobs // all units containing "rate"

count if concept == "coicop" // 804
list if concept == "coicop" & regexm(code, "^CP[0-9][0-9]$"), clean noobs // main COICOP categories

count if concept == "geo" // 4,280
list if concept == "geo" & inlist(code, "EE", "LT", "LV"), clean noobs // Baltics
list if concept == "geo" & regexm(code_lbl, "Eesti"), clean noobs // Estonian regions
```

Example 2: Determine possible combinations of dimensions

- ▶ Often easiest to retrieve part of the dataset and then narrow the query further down (to save time and bandwidth)
- ▶ There is no need to specify **every** single dimension → fix those you can more easily (e.g. specific country, sector, period)
- ▶ Typically can't leave **all** dimensions unspecified → servers refuse too large data queries

```
sdmxuse data ESTAT, dataset(PRC_HICP_MANR) dimensions(..CP00.) clear  
des // freq, unit, coicop, geo + time, value  
ta freq // M  
ta unit // RCH_A  
ta geo // e.g. EE  
sdmxuse data ESTAT, dataset(PRC_HICP_MANR) dimensions(M.RCH_A.CP00.EE) clear // with all dimensions
```

Example 3: Other data providers

```
// ECB
sdmxuse dataflow ECB, clear
count // 211
sdmxuse data ECB, dataset(MFI) dimensions(..100.T.) clear

// OECD

// IMF
sdmxuse dataflow IMF, clear
count // 322
sdmxuse datastructure IMF, dataset(MFS) clear // (FREQ.REF_AREA.INDICATOR)
sdmxuse data IMF, dataset(MFS) dimensions(A.EE.) start(2020) clear
```

SDMX full options

- ▶ data with labels,
- ▶ start/end period,
- ▶ first/last N observations,
- ▶ version control & updates since,
- ▶ output format (XML, JSON, CSV).

See also SDMX 2.1 cheat sheet.

Statistics Estonia API

- ▶ Since mid-2023 RESTful API no longer supported
- ▶ New API
 - ▶ allows data fetching only with **POST** requests
 - ▶ requests in JSON format
 - ▶ online assistance for compiling requests, see example
 - ▶ outputs in JSON, SDMX, PX, CSV or XLSX
 - ▶ see "API juhend" for details
- ▶ Implications for Stata users
 - ▶ native Stata can only handle GET requests
 - ▶ but Stata (16+) talks to Python, which can handle POST requests
 - ▶ Python can also handle more easily XML and JSON

Example 4: Statistics Estonia API

```
// metadata
python
URL = 'https://andmed.stat.ee/api/v1/et/stat/IA001'
data = requests.get(URL).json()
print(data)
end

// data (single-dimension)
python
URL = 'https://andmed.stat.ee/api/v1/et/stat/IA001'
q = {"query": []}
SA = requests.post(URL,json=q).json()
end
```

See STATA_SET4.DO for the complete example.

Choice of tools

- ▶ Stata
 - ▶ -SDMXUSE- very simple but outdated (I have been updating it myself)
 - ▶ -DBNOMICS- might be a better alternative as it gets updates through a central database (but does not cover Statistics Estonia)
 - ▶ native Stata no longer enough to work with Statistics Estonia API
 - ▶ Stata-Python is a powerful combination, but means learning 2 languages
- ▶ R and Python
 - ▶ better native support for various web protocols
 - ▶ potentially more effort to learn
- ▶ Most mainstream software can handle APIs one way or another, but if focus on (macro) statistics rather than econometrics then R or Python could be a more obvious choice

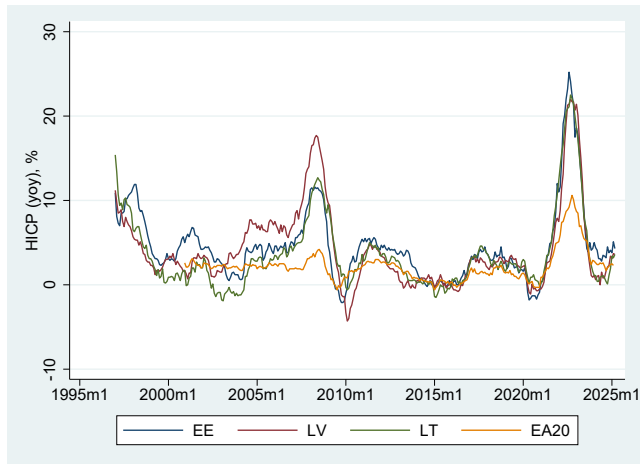
General recommendations for visualisation

See e.g. Schwabish, J. (2024), An Economist's Guide to Visualizing Data, Journal of Economic Perspectives, 28(1): 209-234. DOI

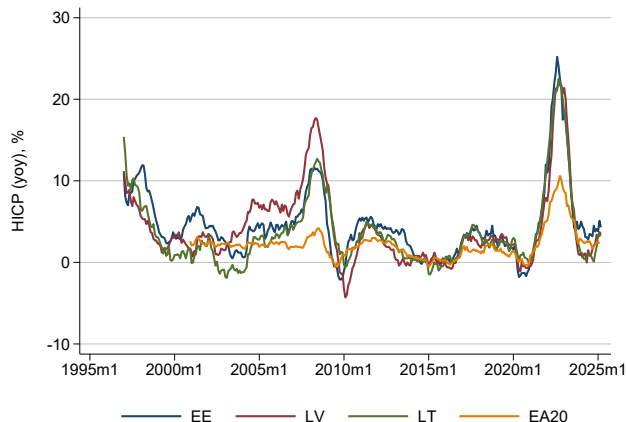
- ▶ Each figure should be stand-alone and self-explanatory
- ▶ Keep things simple – avoid too many data series at once, different types, dual axes etc
- ▶ Minimal design
 - ▶ avoid clutter – no background shading, frames, unnecessary special effects etc
 - ▶ use of colours primarily for (live) presentations, printable publications need to be understandable in black & white

If **repeatedly** used/updated – programmable or at least using templates/shells

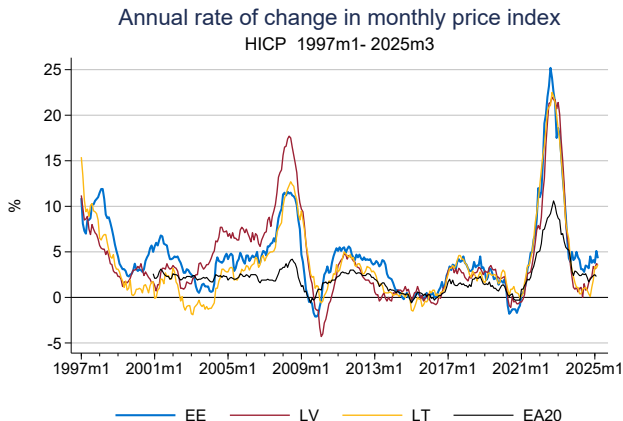
Example 1': Stata default



Example 1: Customised graphics SCHEME to remove clutter



Example 1": Final tweaks



Example 1": Final Stata code

```
// line colours to match flag colours (rgb)
global ee "0 114 206"
global lv "157 34 53" // red, red fabric (119 53 61)
global lt "253 185 19" // yellow

// determine available period
su ym if inlist(geo, "EE", "LV", "LT", "EA20")
local min : display %tm r(min)
local max : display %tm r(max)

// figure
tw (line value ym if geo == "EE", lcolor("${ee}") lwidth(medthick)) ///
   (line value ym if geo == "LV", lcolor("${lv}") lwidth(medium)) ///
   (line value ym if geo == "LT", lcolor("${lt}") lwidth(medium)) ///
   (line value ym if geo == "EA20", lcolor(black) lwidth(medthin)) ///
yline(0, lcolor(black) lwidth(vthin)) ylabel(-5(5)25) ///
tmtick(1997m1(12)2025m1) tlabel(1997m1(48)2025m1) ///
title("Annual rate of change in monthly price index") ///
subtitle("HICP 'min'-'max'") xtitle("") ytitle("%") ///
legend(rows(1) order(1 "EE" 2 "LV" 3 "LT" 4 "EA20" ))
```

Where Excel figures are too much pain or not supported

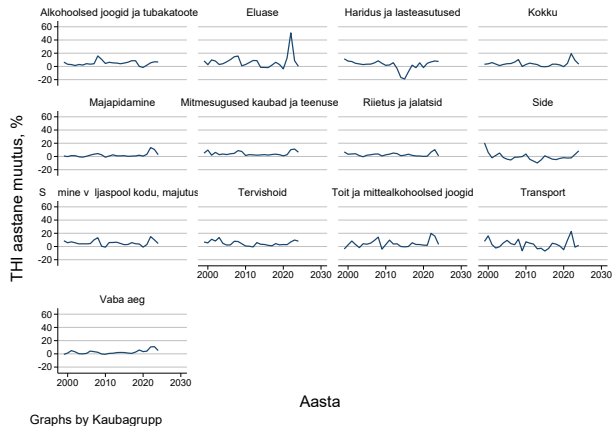
- ▶ Scatter plots with marker labels
- ▶ Graphs by subgroups (in MxN outline)
- ▶ Combine graphs
- ▶ Thematic maps
- ▶ etc

Caveats:

- ▶ It is possible that Copilot can now do it for you, but built-in shortcomings still
- ▶ One could also rely on VBA coding (+ Copilot) to program these tasks, but investing in R and/or Python skills is likely more valuable

Example 4: Figure by subgroups

tw line value Aasta, by(Kaubagrupp)



The Curse of Word

- ▶ Everyone can use (at the beginner level)
- ▶ Advanced use potentially more demanding than e.g. LaTeX
 - ▶ Document templates and text styles often ignored or underused
 - ▶ Captions and TOC
 - ▶ Linking figures/tables from external files
 - ▶ etc

Papers, reports and slides

- ▶ Link to external tables/figures from Word/Excel/Powerpoint
- ▶ Export directly to Word/Excel/Powerpoint

Stata offers many possibilities, but R is even more capable – a fully integrated approach from data fetching to complete final documents.

'AI' – experiences, current uses and training needs

Category	Task	Usefulness
Ideation and Feedback	Brainstorming	●
	Feedback	◐
	Providing counterarguments	◐
Writing	Synthesizing text	●
	Editing text	●
	Evaluating text	●
	Converting hand-written equations	○
	Generating catchy titles & headlines	●
	Generating tweets to promote a paper	●
	Generating presentation slides	●
Background Research	Summarization	●
	Literature Research	◐
	Formatting References	●
	Translating Text	●
	Explaining Concepts	◐

The third column reports my subjective rating of LLM capabilities as of April 2024:

- : experimental; results are inconsistent and require significant human oversight
- ◐: useful; requires oversight but will likely save you time
- : highly useful; incorporating this into your workflow will save you time

Category	Task	Usefulness
Coding	Writing code	●
	Explaining code	●
	Translating code	●
	Debugging code	●
Data Analysis	Locating data sources	●
	Creating figures	●
	Extracting data from text	●
	Reformatting data	●
	Classifying and scoring text	●
	Extracting sentiment	●
	Simulating human subjects	●
Math	Setting up models	●
	Deriving equations	○
	Explaining models	●

The third column reports my subjective rating of LLM capabilities as of April 2024:

○: experimental; results are inconsistent and require significant human oversight

●: useful; requires oversight but will likely save you time

●: highly useful; incorporating this into your workflow will save you time

Examples related to data processing and modelling

- ▶ Fetch and process data from the web (e.g. airline seat sizes)
- ▶ Help to develop
 - ▶ complex spreadsheet formulas (Excel)
 - ▶ program code/data scripts (Python, R, Stata?!)
 - ▶ text syntax (LaTeX)
- ▶ Assist with methodological questions and interpretation of results
- ▶ etc

Q & A

Aitäh!

Küsimused ja kommentarid?

alari.paulus@eestipank.ee

nicolas.reigl@eestipank.ee

** Special thanks to SPJ for enduring inspiration!*

Advanced functionality in Excel

- ▶ Referencing (naming, \$, ADDRESS, INDIRECT)
- ▶ Link and combine data (VLOOKUP, HLOOKUP, MATCH, CONCATENATE)
- ▶ Conditional formatting with formulas
- ▶ Summarise data with functions (COUNTIF, SUMIF, SUMPRODUCT)
- ▶ Summarise and filter data with pivot tables
- ▶ Date and time functions
- ▶ (Matrix functions)

Tools for research: referencing and editors

- ▶ Bibliography reference manager – [Zotero](#) (free)
 - ▶ central database – enter/edit references only once
 - ▶ add items with a single click
 - ▶ flexible structuring (tagging) and database-wide search
 - ▶ compile bibliography automatically (Word, LaTeX)
 - ▶ share and collaborate
- ▶ LaTeX editor – [Overleaf](#) (free)
 - ▶ online tool, no need for local installations
 - ▶ easy to use – dual view (code vs compiled), also visual editor
 - ▶ share and collaborate (track changes)

Tools for research: notes

- ▶ Organising notes – [OneNote](#) (part of Office)
 - ▶ everything in one place – search easily across projects etc
 - ▶ multi-level structure (notebooks > sections > pages)
 - ▶ multi-format (text, tables, figures)
 - ▶ copy-paste with automatic sources
 - ▶ cloud storage – backup and access from multiple devices
 - ▶ share and collaborate